



MOMENTO III

Autores: *Maria Valentina Quiroga Alzate, Juan Felipe Orozco Londoño*

Informática II

*Departamento de Ingeniería Electrónica y de Telecomunicaciones
Universidad de Antioquia*

0.1. Capítulos escogidos

Para el desarrollo del videojuego se seleccionaron los episodios comprendidos entre el número 70 y el 76 de la serie *Dragon Ball*, correspondientes a la saga del Palacio de Uranai Baba. Esta elección se fundamenta en la riqueza de combates creativos, así como en la variedad de personajes y escenarios, los cuales resultan altamente adaptables al diseño de niveles para un videojuego sencillo. Además, las características de esta saga permiten implementar físicas jugables en Qt de manera coherente con la lógica de juego y con posibilidades técnicas accesibles.

0.2 Detalles adicionales respecto a los momentos I y II

0.2.1 Confirmación y consolidación de modelos físicos

En los momentos anteriores se propusieron tres modelos físicos fundamentales: el salto parabólico, el deslizamiento con fricción reducida y el empuje energético. Durante la implementación final, estos modelos fueron confirmados, refinados y, en algunos casos, reemplazados por versiones más adecuadas al contexto jugable y técnico del sistema.

Salto parabólico: Se implementó exitosamente mediante la clase `SaltoParabolico`. Este modelo permitió ajustar la velocidad inicial, la gravedad y la duración máxima del salto. Fue activado mediante eventos de teclado y ejecutado

con un temporizador `QTimer` que actualiza la posición del personaje *frame a frame*. Se permitió al jugador evadir ataques del Hombre Invisible y moverse por el escenario de forma fluida.

Movimiento oscilatorio: En lugar de implementar el deslizamiento con fricción, se decidió utilizar un modelo más dinámico: el movimiento oscilatorio armónico, programado en la clase `MovimientoOscilatorio`. Este se aplicó al enemigo invisible, generando trayectorias más suaves en el eje horizontal y aportando variabilidad a sus ataques para que no sean predecibles. La ecuación implementada corresponde a una función coseno que simula el desplazamiento armónico simple.

Empuje energético por colisión: Conservado e integrado como parte de las físicas en ambos niveles. Cuando Gokú o los enemigos reciben un impacto (poder o colisión de ataque), se produce un desplazamiento brusco simulado mediante `moveBy()` y temporizadores para efectos visuales. Esta física intensifica el riesgo en el Nivel 2, especialmente cerca de los bordes de la plataforma.

0.2.2 Validación de visibilidad y tiempos de ataque

La mecánica del Hombre Invisible fue refinada con precisión para que su aparición siga un patrón jugable, pero impredecible. El enemigo permanece oculto por intervalos de algunos segundos, y aparece durante 3 segundos. Esta lógica fue implementada mediante `QTimer` y un flag

de visibilidad controlado internamente.

Durante las ventanas de visibilidad, se activa la colisión con `collidesWithItem()` y se permite que el ataque de Gokú tenga efecto. Fuera de estas ventanas, el enemigo es intangible y el jugador debe mantenerse atento a que en cualquier momento aparezca su enemigo.

Desde el punto de vista técnico, se utilizaron señales de Qt, `QTimer` y estados booleanos internos para gestionar visibilidad, animación y daño.

0.2.3 Reforzamiento del modelo de colisión y áreas válidas

Se mantuvo el modelo de colisión basado en la superposición de áreas (`collidesWithItem()`) y se extendió con validaciones de área segura (`QRectF`) para determinar condiciones de caída.

En el Nivel 1, el Hombre Invisible activa su zona de colisión únicamente cuando está visible, lo cual se validó mediante un flag `estaVisible()`.

En el Nivel 2, tanto Gokú como el Hombre Enmascarado tienen asociadas zonas válidas de desplazamiento. Si alguno de ellos sale del área `QRectF` predefinida, se activa una condición de caída: el personaje pierde una vida y reaparece en el centro. Si las vidas llegan a cero, se muestra un mensaje de derrota. Estas colisiones se utilizan tanto para ataques como para detectar la salida del área jugable.

0.2.4 Redefinición de objetivos y condiciones de victoria/derrota

Durante la implementación se ajustaron los criterios de éxito y derrota para mejorar la jugabilidad y claridad de objetivos:

Nivel 1: El objetivo era acertar al menos cinco golpes válidos al Hombre Invisible. En la versión final, esto se simplificó: el jugador debe reducir completamente la barra de vida del enemigo antes de quedarse sin corazones. Cada golpe válido causa 20 puntos de daño, y al llegar a 0, se

activa una escena de transición al siguiente nivel. La lógica fue manejada con una barra de energía, vidas y señales de Qt (`nivelCompletado()`).

Nivel 2: El objetivo se mantuvo: reducir la vida del Hombre Enmascarado a cero sin que Gokú caiga fuera de la plataforma. Se añadieron validaciones visuales y lógicas para ambos criterios: el HUD muestra la barra de vida, y las colisiones se evalúan continuamente mediante `QTimer`. Si Gokú cae, pierde una vida; si llega a 0, se muestra una ventana de derrota (`QMessageBox`). Si el enemigo muere, se lanza una escena narrativa como desenlace del juego.

0.2.5 Cambios arquitectónicos y de diseño respecto al Momento II

- Se implementó un sistema flexible de niveles mediante punteros dinámicos a `Nivel1` y `Nivel2` en la clase `Game`, permitiendo transición limpia entre escenas.
- Se modularizó el sistema de físicas y ataques, separando `Poder` y `PoderEnemigo`, y aislando lógicas individuales.
- Se sustituyó el modelo de deslizamiento previsto por una física más impactante y controlada: el empuje energético.
- Se integró la detección de caída como evento automático, vinculado a las coordenadas del personaje y su pertenencia al área segura.
- Se añadieron excepciones controladas para carga de sprites y validación de áreas, cumpliendo el criterio de robustez.

0.2. Descripción detallada de vistas e interacciones

Nivel 1 – El Hombre Invisible

Este nivel se desarrolla en un escenario con vista lateral fija, ambientado en un pasillo del palacio de Uranai Baba. El jugador controla a Gokú, quien se desplaza horizontalmente sobre una superficie continua, con posibilidad de saltar y atacar. La escena se gestiona mediante `QGraphicsScene` y los elementos interactivos se heredan de `QGraphicsItem`.

El enemigo, el Hombre Invisible, permanece oculto la mayor parte del tiempo y aparece brevemente cada cierto intervalo, siguiendo un patrón aleatorio. Su visibilidad está controlada mediante un `QTimer` y una bandera interna que activa su colisión solo durante esas ventanas. Esto obliga al jugador a mantenerse alerta y ejecutar ataques sincronizados.

Durante los momentos en los que el enemigo aparece (aproximadamente 3 segundos), se muestra su sprite parcial mediante una animación sutil. Si el jugador ataca en ese instante y colisiona con él, se registra un golpe válido, reduciendo su energía.

La interfaz de usuario incluye:

- Barra de energía de Gokú y del enemigo.
- Indicador de corazones (vidas), ubicados en la parte superior izquierda para Gokú y superior derecha para el enemigo.
- Señales visuales (efectos de opacidad).

Las físicas implementadas en este nivel son:

- **Salto parabólico**, controlado por la clase `SaltoParabolico`, que calcula la trayectoria vertical bajo gravedad simulada.
- **Movimiento oscilatorio** para el enemigo, con trayectorias suaves en

el eje horizontal, mediante la clase `MovimientoOscilatorio`.

Además, se ha añadido manejo dinámico al cargar los sprites principales. Si Gokú recibe daño, se activa una animación temporal con opacidad reducida y sprite de golpe. Si pierde todas las vidas, se muestra una ventana de derrota. Al derrotar al Hombre Invisible, se muestra un mensaje de victoria y el sistema transiciona automáticamente al Nivel 2.

Los controles activos son:

- Flechas izquierda y derecha para moverse.
- Barra espaciadora para saltar.
- Tecla X para atacar.



Figura 1: Primer nivel: combate contra el Hombre Invisible.

Nivel 2 – El Hombre Enmascarado (Son Gohan)

El segundo nivel presenta una vista superior (*top-down*) sobre una plataforma flotante. A diferencia del Nivel 1, aquí los movimientos son en ocho direcciones y el jugador puede ser expulsado del escenario. El fondo representa el cielo, reforzando el riesgo de caída.

El combate se realiza contra el Hombre Enmascarado (Son Gohan), quien se desplaza autónomamente y lanza poderes dirigidos al jugador cada cierto intervalo. Tanto Gokú como el enemigo pueden colisionar entre sí y generar empujes que los desplacen peligrosamente hacia los bordes.

Se utiliza un área segura definida mediante un `QRectF`, la cual es evaluada constantemente para detectar si algún personaje ha salido de ella. Si Gokú cae, pierde una vida y reaparece en el centro de la plataforma. Si pierde todas las vidas, se muestra una ventana de derrota. Si Gohan pierde toda su energía, se muestra una escena de victoria y cierre narrativo.

La física implementada incluye:

- **Empuje energético** al recibir impacto de poderes o por colisión directa durante ataque.
- **Detección de caída** por fuera del área segura.
- **Animación de daño** con opacidad temporal.

Elementos nuevos integrados:

- HUD con corazones de vida y barras de energía para ambos personajes.
- Detección de caídas mediante `contains(pos())` sobre el área segura.
- Movimiento del enemigo con cambio aleatorio de dirección y sprite.
- Poderes enemigos que apuntan a la posición actual de Gokú.
- Validación de condiciones de victoria y derrota.

Controles activos:

- Flechas arriba, abajo, izquierda y derecha para moverse.
- Tecla X para atacar.



Figura 2: Segundo nivel: combate contra el Hombre Enmascarado.

0.3. Sprites utilizados por nivel

En cumplimiento con los requisitos establecidos para el diseño del videojuego, se han definido y utilizado al menos cuatro sprites distintos por nivel. Estos sprites representan personajes, enemigos, elementos del entorno y efectos visuales relevantes para la jugabilidad. Todos los sprites fueron seleccionados con cuidado para mantener coherencia visual con el estilo de la serie original de Dragon Ball, adaptados al entorno de desarrollo en Qt.

Nivel 1 – El Hombre Invisible

Los sprites utilizados en este nivel son:

- Gokú (`goku.png`), con múltiples animaciones de movimiento, ataque y reacción al daño.

- Enemigo invisible (`enemigo_nivel1.png`), una silueta parcialmente visible durante ataques o intervalos específicos.
- Sprite del corazón de vida (`corazon.png`), representando la vida restante tanto del jugador como del enemigo.
- Poder básico de Gokú (`poder.png`), con movimiento dirigido hacia el enemigo.
- Poder básico de enemigo (`poder_enemigo.png`), con movimiento dirigido hacia el enemigo.



Figura 3: Indicador de vida: corazón.

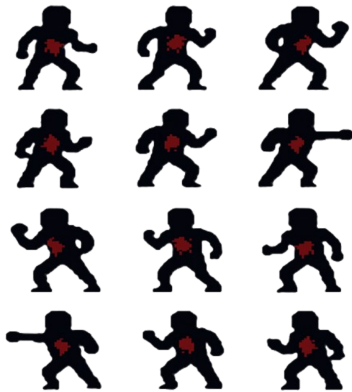


Figura 4: Sprite del Hombre Invisible.

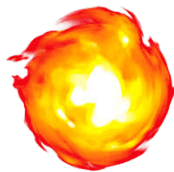


Figura 5: Poder de Goku.

Nivel 2 – El Hombre Enmascarado

Este nivel contiene una nueva ambientación, más dinámica y abierta, por lo que se implementaron sprites distintos y más detallados:

- Enemigo: Hombre Enmascarado (`enemigo_nivel2.png`), con animaciones de desplazamiento y ataque.
- Fondo del nivel 2 (`fondo_nivel2.png`): representa una plataforma flotante en el cielo.
- Efecto visual del poder enemigo (`poder_enemigo.png`), con forma circular y tono púrpura para distinguirlo del poder de Gokú.
- HUD visual que incluye los mismos corazones e indicadores de energía que en el Nivel 1, adaptados a ambos personajes.

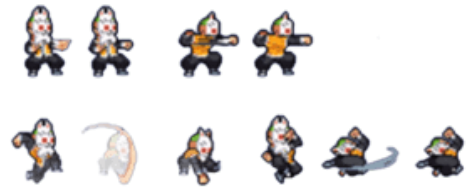


Figura 6: Sprite del Hombre Enmascarado.

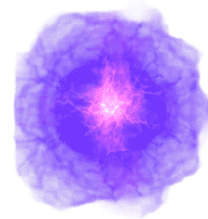
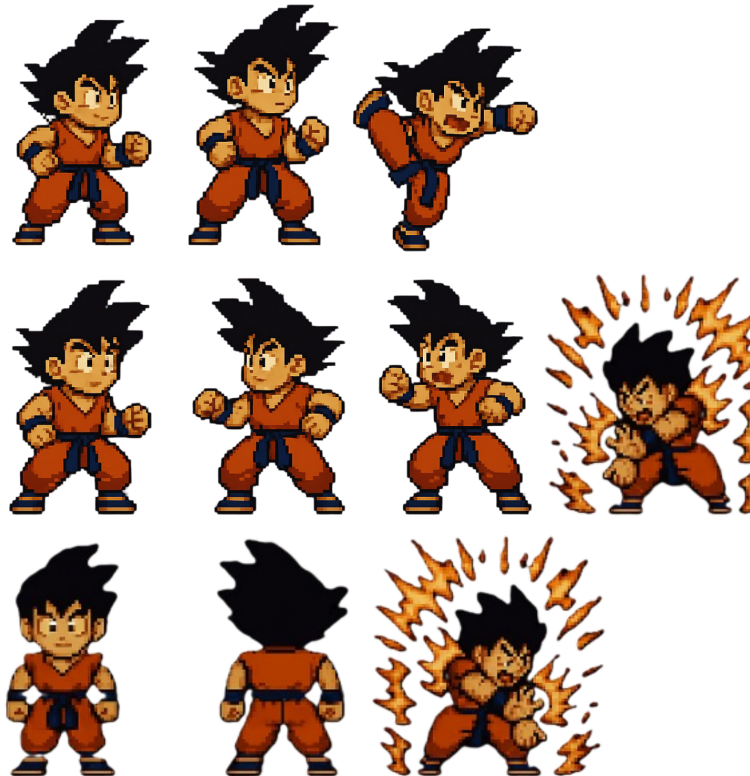


Figura 7: Poder enemigo.

Sprite compartido: Gokú

El sprite de Gokú es compartido entre ambos niveles y cuenta con múltiples cuadros para las siguientes animaciones:

- Movimiento en diferentes direcciones.



- Ataque (golpe y energía).



Figura 8: Sprite principal de Gokú.

- Recibir daño (con cambio de sprite y opacidad).

0.4. Diagrama de clases

0.5. Análisis del Diagrama de Clases

- Transformación y efectos especiales (como aura o energía acumulada).

El diseño del videojuego se basa en una arquitectura modular orientada a objetos, que separa claramente las responsabilidades en diferentes capas: control, lógica del juego, representación

visual y físicas. El diagrama de clases presentado refleja esta organización mediante una jerarquía clara de clases, relaciones de herencia, uso de composición y encapsulamiento de atributos clave.

1. Capa de personajes y enemigos

Jugador es la clase principal controlada por el usuario. Contiene múltiples atributos relacionados con su estado interno: **vidas**, **energía**, **golpesAcertados**, **sprites** y **temporizadores**. Su comportamiento se extiende con métodos como **mover()**, **saltar()**, **atacar()**, **recibirDanio()** y control de físicas a través del objeto **SaltoParabolico**.

Los enemigos derivan de la clase abstracta **Enemigo**, que define las posiciones (**x**, **y**), velocidades y una referencia a una física genérica (**Fisica***). Esta clase base impone dos métodos puros: **mover()** y **actualizarFisica()**, que son implementados de forma distinta en sus subclases:

- **HombreInvisible**: incluye control de visibilidad con un **QTimer**, sprites parciales, barra de energía y lógica propia de aparición/desaparición. Además, tiene un puntero a la vista para emitir señales como **nivelCompletado()**.
- **HombreEnmascarado**: incluye manejo de energía, verificación de caída por **QRectF**, animaciones de empuje y ataques, y la lógica de victoria final del juego.

Ambos enemigos contienen listas de corazones representadas con **QList<QGraphicsPixmapItem*>**, lo cual permite mostrar y actualizar gráficamente la cantidad de vidas restantes.

2. Capa de físicas

La clase abstracta **Fisica** define la estructura base para las físicas del juego, con métodos

virtuales puros que se especializan según el contexto. Las clases derivadas incluyen:

- **MovimientoOscilatorio**: aplicada al **Hombre Invisible** para generar movimiento sinusoidal horizontal.
- **SaltoParabolico**: aplicada al **Jugador**, simula trayectorias balísticas controladas con temporizador.

Ambas físicas dependen de parámetros ajustables como amplitud, velocidad inicial, gravedad o duración máxima, y operan sobre las variables de posición y velocidad de los personajes.

3. Capa de poderes

El sistema de ataque se compone de dos clases:

- **Poder**: proyectil lanzado por el jugador. Tiene dirección, velocidad, escena de referencia y un **QTimer** que lo mueve cada ciclo.
- **PoderEnemigo**: proyectil lanzado por los enemigos. Hereda la lógica de movimiento e incluye un puntero directo al jugador para aplicar daño.

Ambos poderes implementan efectos visuales como opacidad y desplazamientos simulando empujes, y son destruidos automáticamente si salen de la pantalla o impactan.

4. Capa de niveles

Las clases **Nivel1** y **Nivel2** encapsulan la lógica de cada escena jugable:

- **Nivel1**: gestiona la aparición del **Hombre Invisible**, su visibilidad y los controles básicos de Gokú.
- **Nivel2**: maneja al **Hombre Enmascarado**, la validación del área segura (**QRectF**) y los empujes por colisión.

Ambas clases contienen una instancia de `Jugador*`, un enemigo, una escena (`QGraphicsScene*`) y varios temporizadores para las mecánicas dinámicas.

5. Clase central de control

La clase `Game` actúa como controlador principal. Contiene referencias a los niveles, a la escena actual, y métodos para iniciar el juego (`start()`) y cambiar de nivel (`cambiarANivel2()`). También redirige los eventos de teclado hacia el jugador correspondiente según el nivel activo.

6. Relaciones de herencia y composición

- **Herencia:** `Enemigo` → `HombreInvisible`, `HombreEnmascarado`. También `Fisica` → `SaltoParabolico`, `MovimientoOscilatorio`.
- **Composición:** `Jugador` contiene un `SaltoParabolico*`; cada enemigo tiene su propia barra de energía, corazones y sprites.
- **Agregación:** los niveles agregan al jugador y enemigos dentro de una escena gráfica compartida, permitiendo modularidad y reutilización de objetos.

El diagrama de clases refleja un diseño estructurado para la reutilización de código y facilidad de la incorporación de nuevos niveles, enemigos o físicas. Se respetan a la programación orientada a objetos, como encapsulamiento, modularidad, herencia y polimorfismo. Además, el uso de punteros dinámicos y señales de Qt garantiza una comunicación eficiente entre objetos que es importante en una aplicación interactiva como un videojuego.

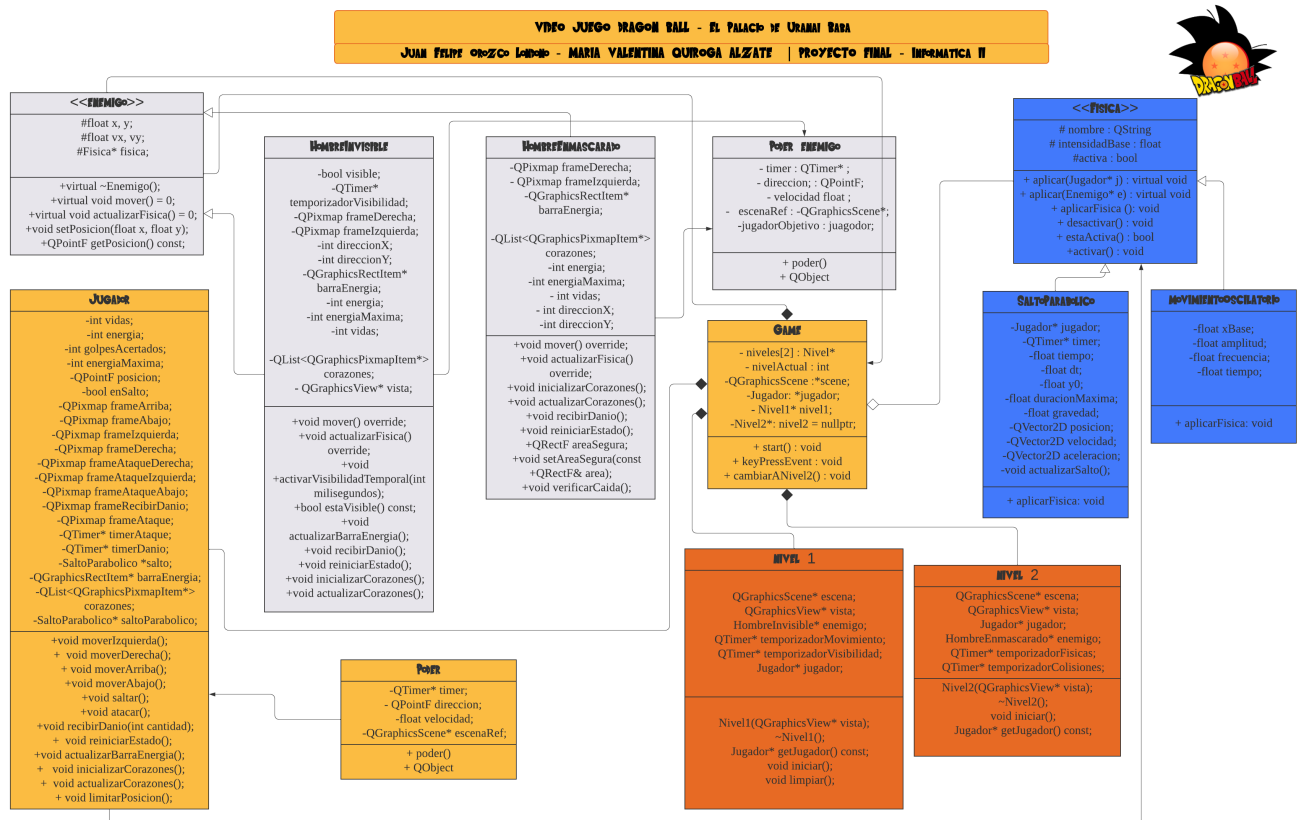


Figura 9: Diagrama de clases.

Bibliografía

Kanzenshuu, *Dragon Ball Episode 076*, Kanzenshuu, [Online]. Available: <https://www.kanzenshuu.com/episode/db/eps-076/>.

Pierre2011amkpopo, *Grandpa Gohan sprite sheet - Dragon Ball classic*, DeviantArt, [Online]. Available: <https://www.deviantart.com/pierre2011amkpopo/art/Grandpa-gohan-sprite-sheet-Dragon-ball-classic-1177609805>.